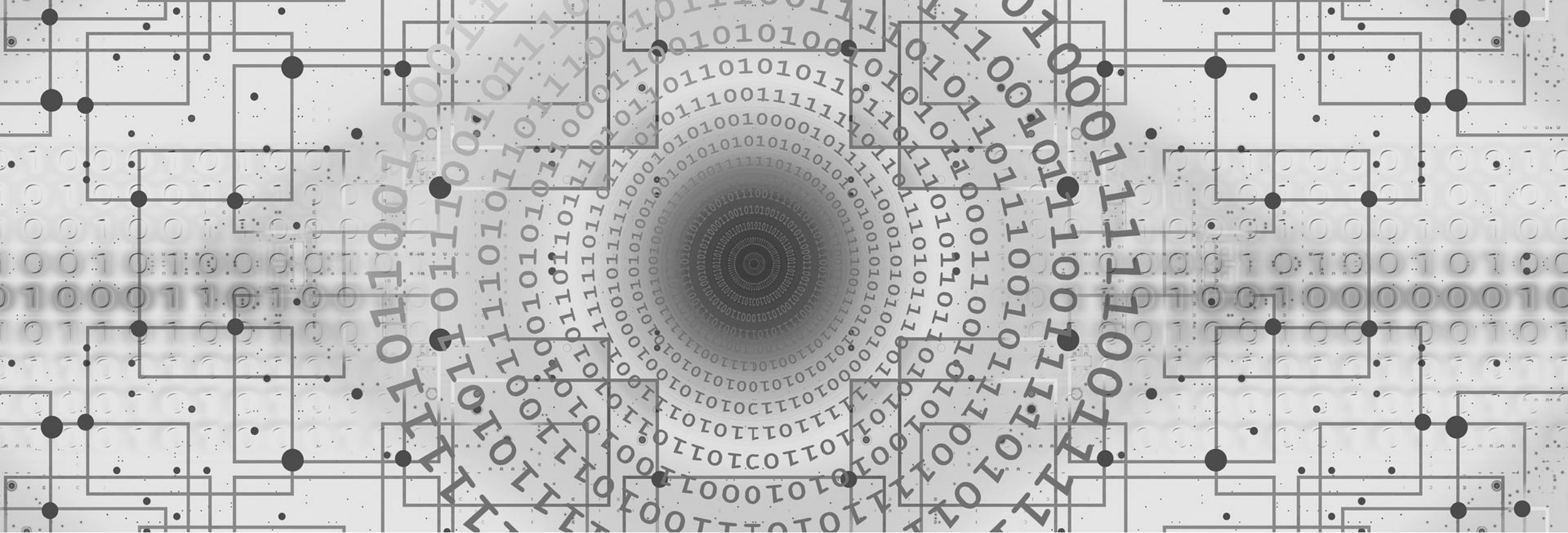


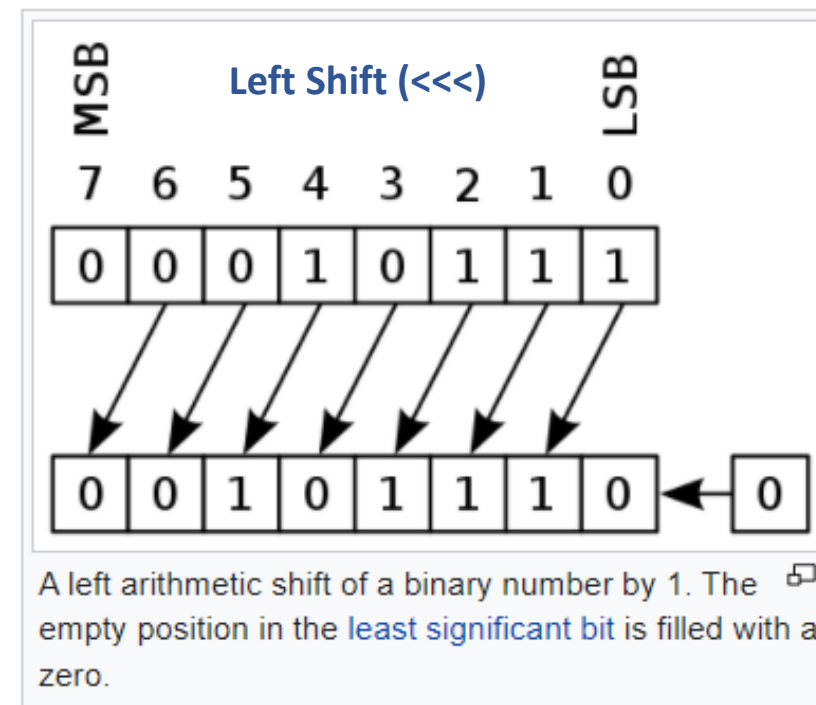
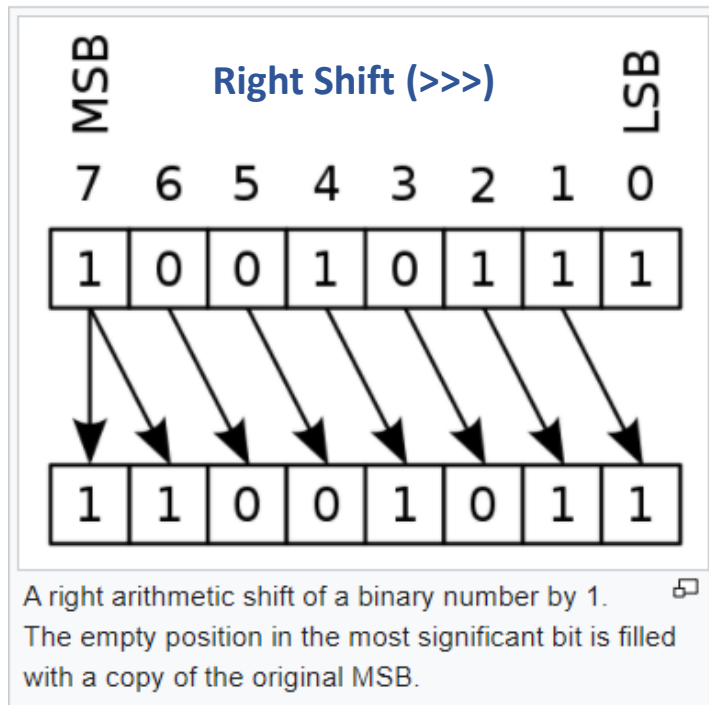


Booth Multiplier (Signed Integer Multiplication)

ECE-111 Advanced Digital Design Project

What is Arithmetic Shift Operation

- ❑ An **arithmetic shift** is a shift operator, sometimes termed a **signed shift** (though it is not restricted to signed operands).
- ❑ The two basic types are the **arithmetic left shift** ($\ll$) and the **arithmetic right shift** ($\gg$)
 - For binary numbers is a bitwise operation that shifts all of the bits of its operand
 - Every bit in the operand is simply moved a given number of bit positions, and the vacant bit-positions are filled in
 - Instead of being filled with all 0s, as in logical shift, when shifting to the right, the leftmost bit (usually the sign bit in signed integer representations) is replicated to fill in all the vacant positions (this is a kind of sign extensions)



Negative Decimal Representation in Binary Two's Complement Form

❑ Procedure to represent a negative decimal value (signed integer) in binary 2's complement form

Two's complement is achieved by:

- Step 1: starting with the equivalent positive number.
- Step 2: inverting (or flipping) all bits – changing every 0 to 1, and every 1 to 0;
- Step 3: adding 1 to the entire inverted number, ignoring any **overflow**. Accounting for overflow will produce the wrong value for the result.

For example, to calculate the **decimal** number **-6** in binary:

- Step 1: +6 in decimal is **0110** in binary; the leftmost significant bit (the first 0) is the **sign** (*just 110 in binary would be -2 in decimal*).
- Step 2: flip all bits in **0110**, giving **1001**.
- Step 3: add the place value 1 to the flipped number **1001**, giving **1010**.

To verify that **1010** indeed has a value of **-6**, add the place values together, but *subtracting* the sign from the final calculation.

Because that the first significant digit is the number sign it must be subtracted to produce the correct result: **1010** = $(1 \times -2^3) + (0 \times 2^2) + (1 \times 2^1) + (0 \times 2^0) = 1 \times -8 + 0 + 1 \times 2 + 0 = -6$.

Bits:	1	0	1	0
Decimal bit value:	-8	4	2	1
Binary calculation:	(1×-2^3)	(0×2^2)	(1×2^1)	(0×2^0)
Decimal calculation:	1 × -8	0	1 × 2	0

Two's complement 4 bit integer values

Two's complement	Decimal
0111	7.
0110	6.
0101	5.
0100	4.
0011	3.
0010	2.
0001	1.
0000	0.
1111	-1.
1110	-2.
1101	-3.
1100	-4.
1011	-5.
1010	-6.
1001	-7.
1000	-8.

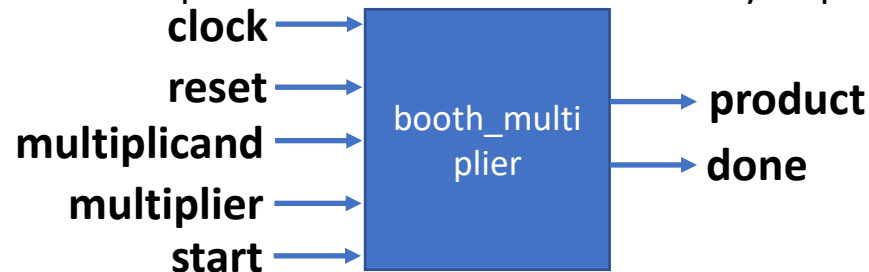
Homework-6c : Booth Multiplier

❑ Develop SystemVerilog RTL model for N-bit Booth Multiplier

- Develop Finite state machine and state transition diagram for Booth multiplier algorithm to support signed decimal number multiplication. For example : $-3 \times -4 = 12$, $-4 \times 5 = -20$, $6 \times -3 = -18$, $7 \times 5 = 35$ all variants with multiplier and multiplicand being negative and positive integer values should be supported.
- FSM coding style recommended : Single always block with non-blocking assignment statements within always block.
- Synthesize booth multiplier design for parameter N=4 and run simulation using testbench provided
- Review synthesis results (resource usage and RTL netlist/schematic)
- Review input and output signals in simulation waveform.
- **Note : FSM code framework is provided in Lab folder with comments to help develop the code.**
- **Testbench provided has built in checker to ensure design output is expected. See messages in Modelsim transcript window when performing simulation**

❑ Assume below mentioned Primary Ports for booth_multiplier module

- **Input** clock, reset : posedge clock and asynchronous posedge reset
- **Input** start : 1 cycle pulse generated. Indicates to FSM to start multiplication operation
- **Input** logic[N-1:0] multiplicand, multiplier : Multiplicand and Multiplier inputs to Integer Multiplier
- **Output** logic[(2*N)-1:0] product : Result of multiplication includes carry bit as MSB bit
- **Output** logic done : Indicates that product is available. This is one cycle pulse generated by FSM.

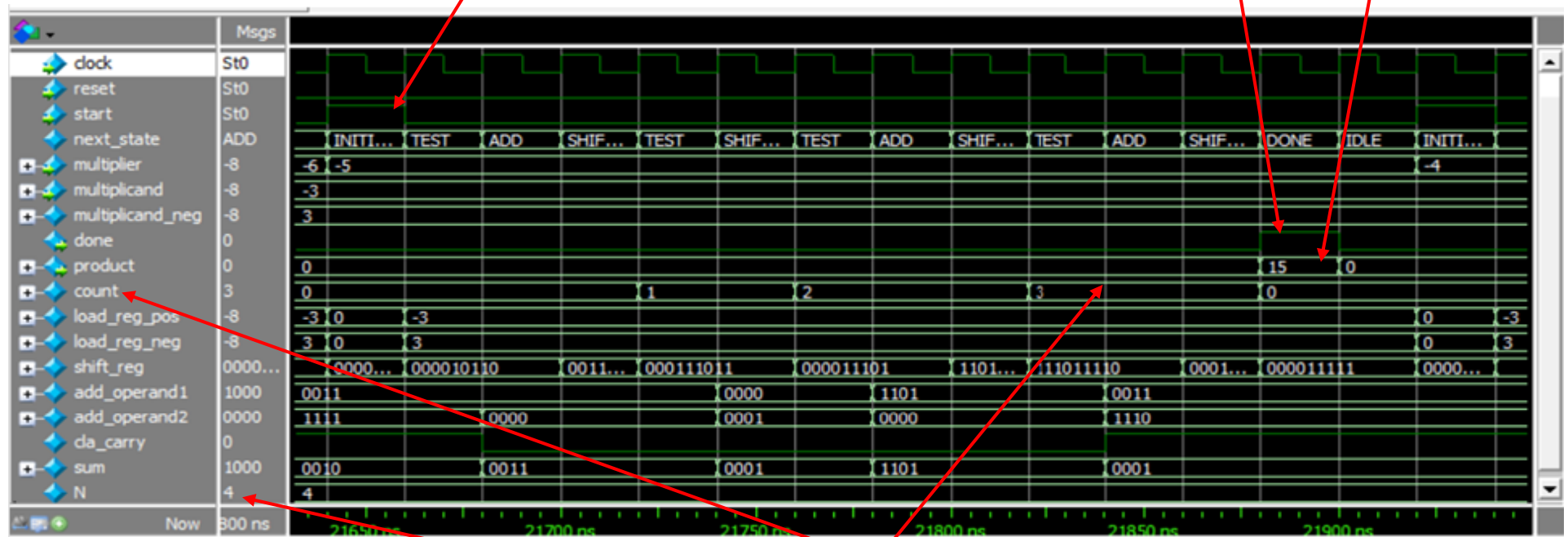


Homework-6c : Booth Multiplier

□ Report should include :

- SystemVerilog FSM design code and State transition diagram (snapshot of hand drawn diagram or Quartus auto-generated diagram, either is acceptable)
- Synthesis resource usage and schematic generated from RTL netlist viewer
- Simulation snapshot and explain simulation result to confirm RTL model developed works as a integer multiplier

□ Reference Simulation Snapshot



Homework-6c : Booth Multiplier

- ❑ Reference transcript window snapshot. Since total 256 combination values of multiplicand and multiplier are tested in testbench, transcript window will be long. And to run a test in Modelsim transcript window, next to **VSIM command "run -all"** and hit enter. See below :

```
Transcript
VSIM 192> run -all
# GetModuleFileName: The specified module could not be found.
#
#
# time=250000  Multiplicand= -8  Multiplier= -8  Product=  64  Done=1 : Correct Result
#
# time=530000  Multiplicand= -8  Multiplier= -7  Product=  56  Done=1 : Correct Result
#
# time=810000  Multiplicand= -8  Multiplier= -6  Product=  48  Done=1 : Correct Result
#
# time=1090000 Multiplicand= -8  Multiplier= -5  Product=  40  Done=1 : Correct Result
#
# time=1330000 Multiplicand= -8  Multiplier= -4  Product=  32  Done=1 : Correct Result
```

- ❑ Ensure, at the end of the test, "Error Count = 0, Test Passed !" is printed. This indicates FSM code functions as per the expectation

```
# time=65490000 Multiplicand=  7  Multiplier=  3  Product=  21  Done=1 : Correct Result
#
# time=65750000 Multiplicand=  7  Multiplier=  4  Product=  28  Done=1 : Correct Result
#
# time=66050000 Multiplicand=  7  Multiplier=  5  Product=  35  Done=1 : Correct Result
#
# time=66310000 Multiplicand=  7  Multiplier=  6  Product=  42  Done=1 : Correct Result
#
# time=66570000 Multiplicand=  7  Multiplier=  7  Product=  49  Done=1 : Correct Result
#
# Error Count = 0, Test Passed !
#
# ** Note: $finish      : C:/projects/systemverilog/booth_multiplier/booth_multiplier_testbench.sv(102)
#   Time: 66800 ns  Iteration: 0  Instance: /booth_multiplier_testbench
# 1
# Break in Module booth_multiplier_testbench at C:/projects/systemverilog/booth_multiplier/booth_multiplier_testbench.sv line 102
VSIM 193>
```

Note : In homework-6c report students can provide part of modelsim transcript snapshot with Test Passed message. Full transcript snapshot is not required.

Booth Method to Perform Signed Integer Multiplier

- ❑ Let's perform multiplication of signed numbers : $-5 \times -3 = 15$. Where Multiplicand is -5 and Multiplier = -3
 - Multiplicand -5 can be represented in **4-bit binary** as : $-M = -5$ (4'b0101) and M (2's complement form) = 4'b1011 (which is decimal 11)
 - Multiplier -3 can be represented in 2's complement form as : 4'b1101 in 4-bit binary (which is decimal 13)
- ❑ N-bit Multiplier has **N-stages** of SHIFT and ADD round of computation. It has Extra bit added which is initialized to 0
- ❑ If Shift Register LSB[1:0] = 01 perform ADD of Accumulator + Multiplicand and store back to Accumulator. Then perform Arithmetic Right Shift by 1
- ❑ If Shift Register LSB[1:0] = 10 perform ADD of Accumulator + ($-$ Multiplicand) and store back to Accumulator. Then perform Arithmetic Right Shift by 1
- ❑ If Shift Register LSB[1:0] = 00 or 11 perform Arithmetic Right Shift by 1

Shift Register[(2*N):0] = {Accumulator[N-1:0], Multiplier[N-1:0], Extra Bit}

	Stage Count#	Accumulator	Multiplier	Extra bit	Operation Performed
	NA	0 0 0 0	1 1 0 1	0	Initialize
acc + (– multiplicand) 0000 + 0101 = 0101	0	0 1 0 1	1 1 0 1	0	ADD
		0 0 1 0	1 1 1 0	1	SHIFT >>> 1
acc + (multiplicand) 0010 + 1011 = 1101	1	1 1 0 1	1 1 1 0	1	ADD
		1 1 1 0	1 1 1 1	0	SHIFT >>> 1
acc + (– multiplicand) 1110 + 0101 = 0011	2	0 0 1 1	1 1 1 1	0	ADD
		0 0 0 1	1 1 1 1	1	SHIFT >>> 1
	3	0 0 0 0	1 1 1 1	1	DONE

Note: Carry out from addition of accumulator and Multiplicand is discarded

Stop once stage count == N-1 reached

Product = 8'b0000_1111 = 15 decimal value

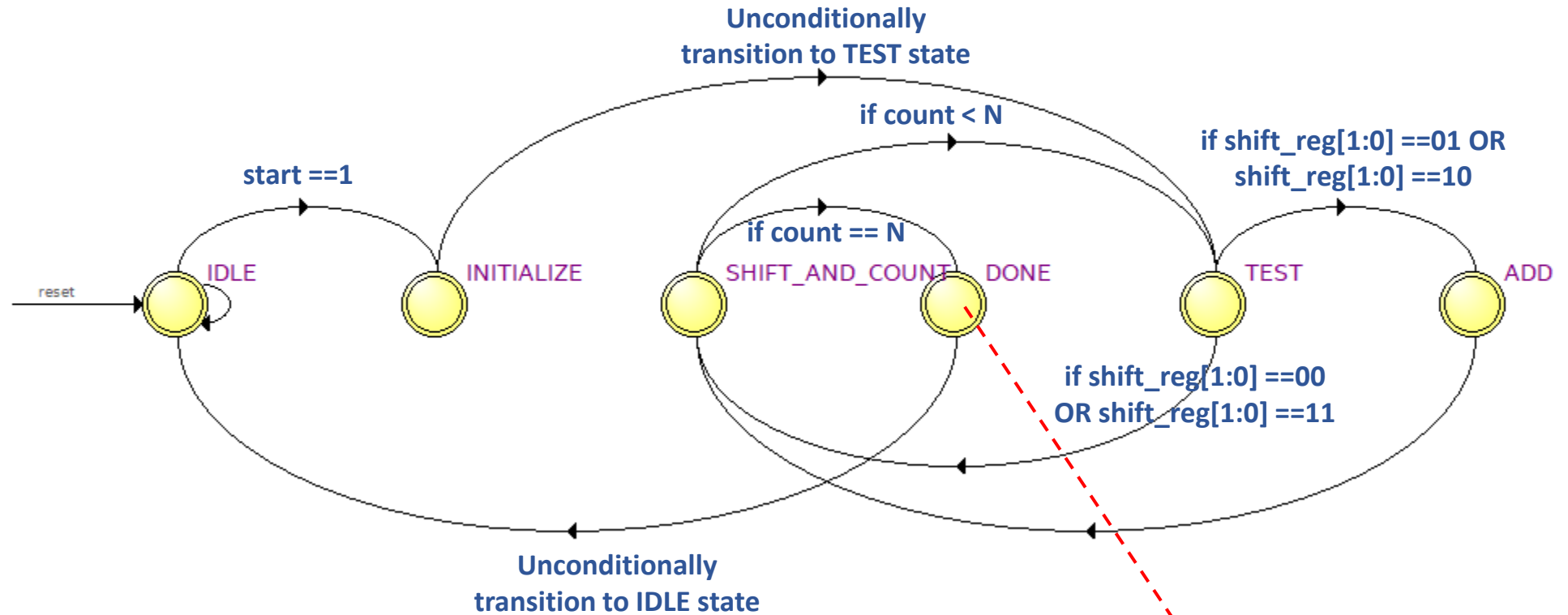
Booth Multiplier Algorithm Summary

❑ Multiplication Process and FSM States :

- **IDLE:** Wait in this state until Start=1. Then move to INITIALIZE state if input signal Start==1
- **INITIALIZE:** Multiplicand, $-$ Multiplicand and Multiplier are loaded into a positive load register, negative load register and a shift register, respectively.
- **TEST:** The LSB in the shift register which contains the multiplier is tested to decide the next state. If shift register LSB[1:0] is '01' or '10', then next state is to **ADD** otherwise next state is to **SHIFT_AND_COUNT**
- **ADD:** If LSB[1:0] is '01', the adder adds previous stage add result with Multiplicand, if LSB[1:0] is '10', the Adder adds previous stage add result with $-$ Multiplicand. The result is stored to the accumulation result, back to shift register and then the state machine transits to **SHIFT_AND_COUNT** state
- **SHIFT_AND_COUNT:** If shift register content is right shifted by 1 bit position. MSB of shift register is sign extended (Read about >>> and <<< operators in previous pages of this document)
- When the stage counter reaches to N, then next state is **DONE** stage otherwise next state is **TEST** state for next stage ADD/SHIFT operation
- **DONE:** Done signal is asserted to '1' and specific bits of shift register content is sent to 'product' output signal

Homework-6b : Signed Integer Multiplier

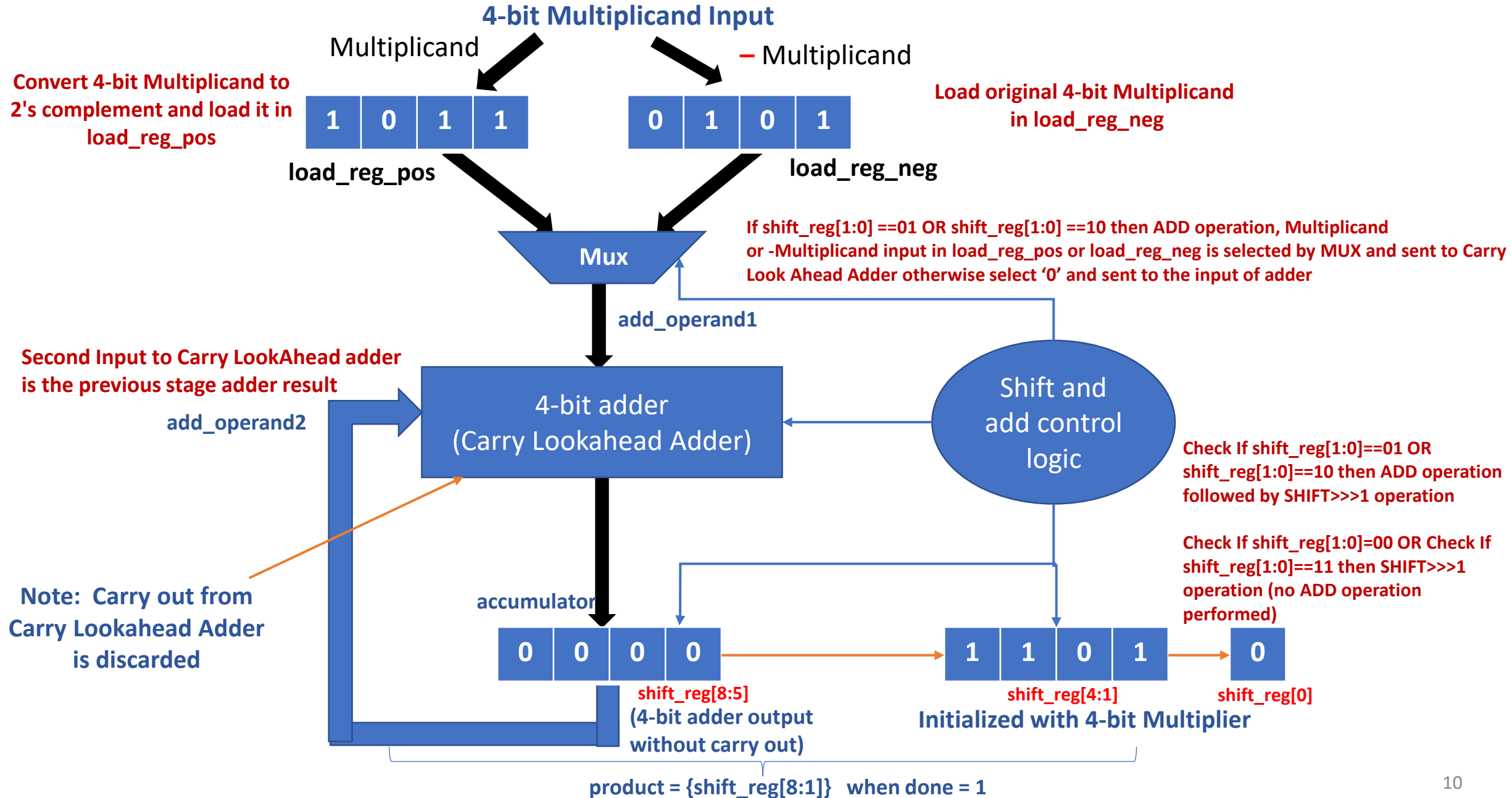
❑ Quartus Generated State Machine Diagram and State Table for reference purpose



Note : When FSM transition to DONE state, load "shift_reg" value to "product"
And set done = 1 from separate logic using assign statement

	Source State	Destination state	Condition
1	ADD	SHIFT_AND_COUNT	
2	DONE	IDLE	
3	IDLE	IDLE	{!start}
4	IDLE	INITIALIZE	{start}
5	INITIALIZE	TEST	
6	SHIFT_AND_COUNT	TEST	{count != N-1}
7	SHIFT_AND_COUNT	DONE	{count == N-1}
8	TEST	SHIFT_AND_COUNT	{shift_reg[1:0] == 00 shift_reg[1:0] == 11}
9	TEST	ADD	{shift_reg[1:0] == 01 shift_reg[1:0] == 10}

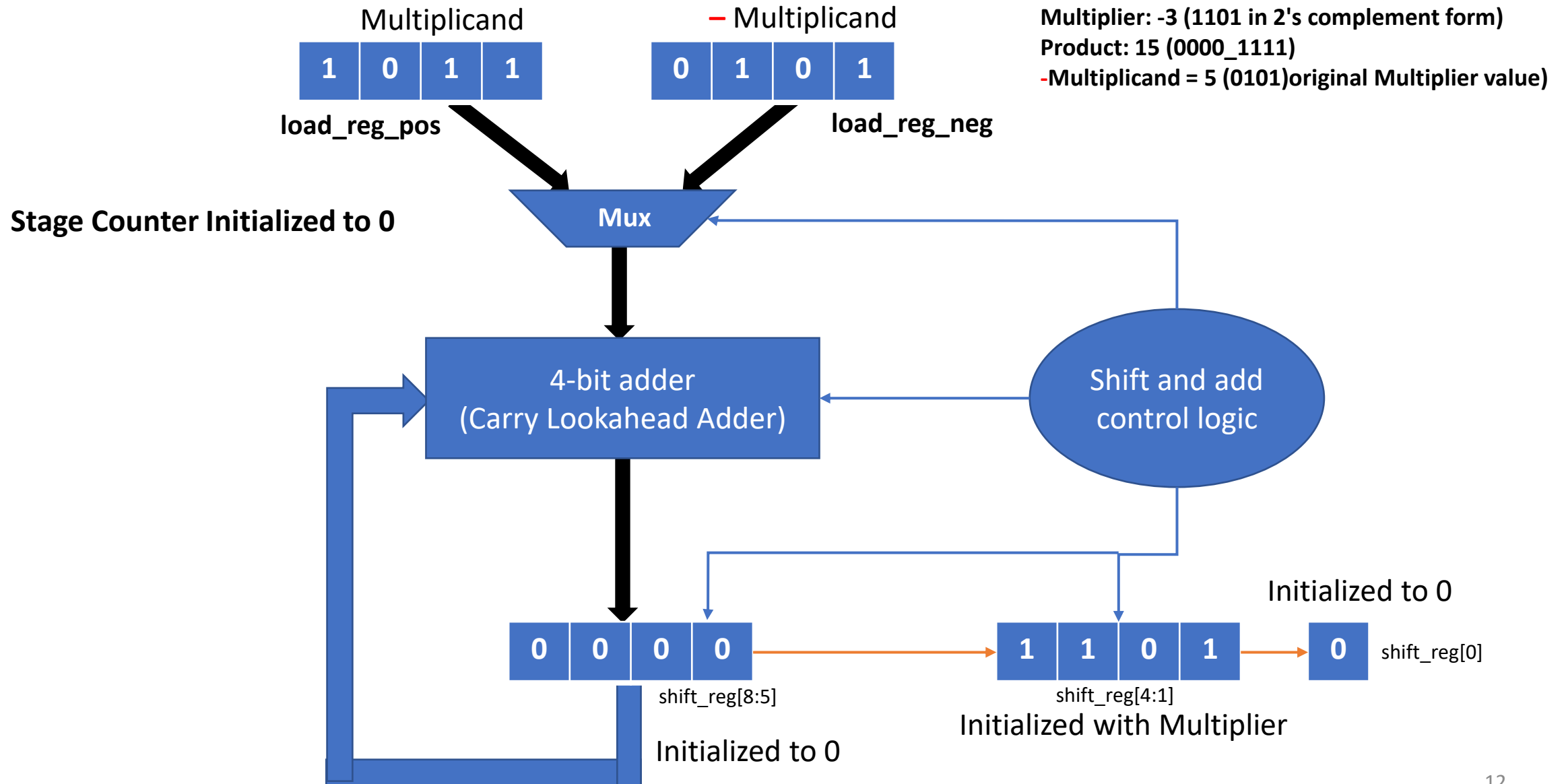
Booth Multiplier Block Diagram



**Let's do simulation of 4-bit Booth Multiplier
using SHIFT and ADD Multiplier Algorithm**

Booth Multiplier Algorithm Example Steps

Count-0 : Initialize

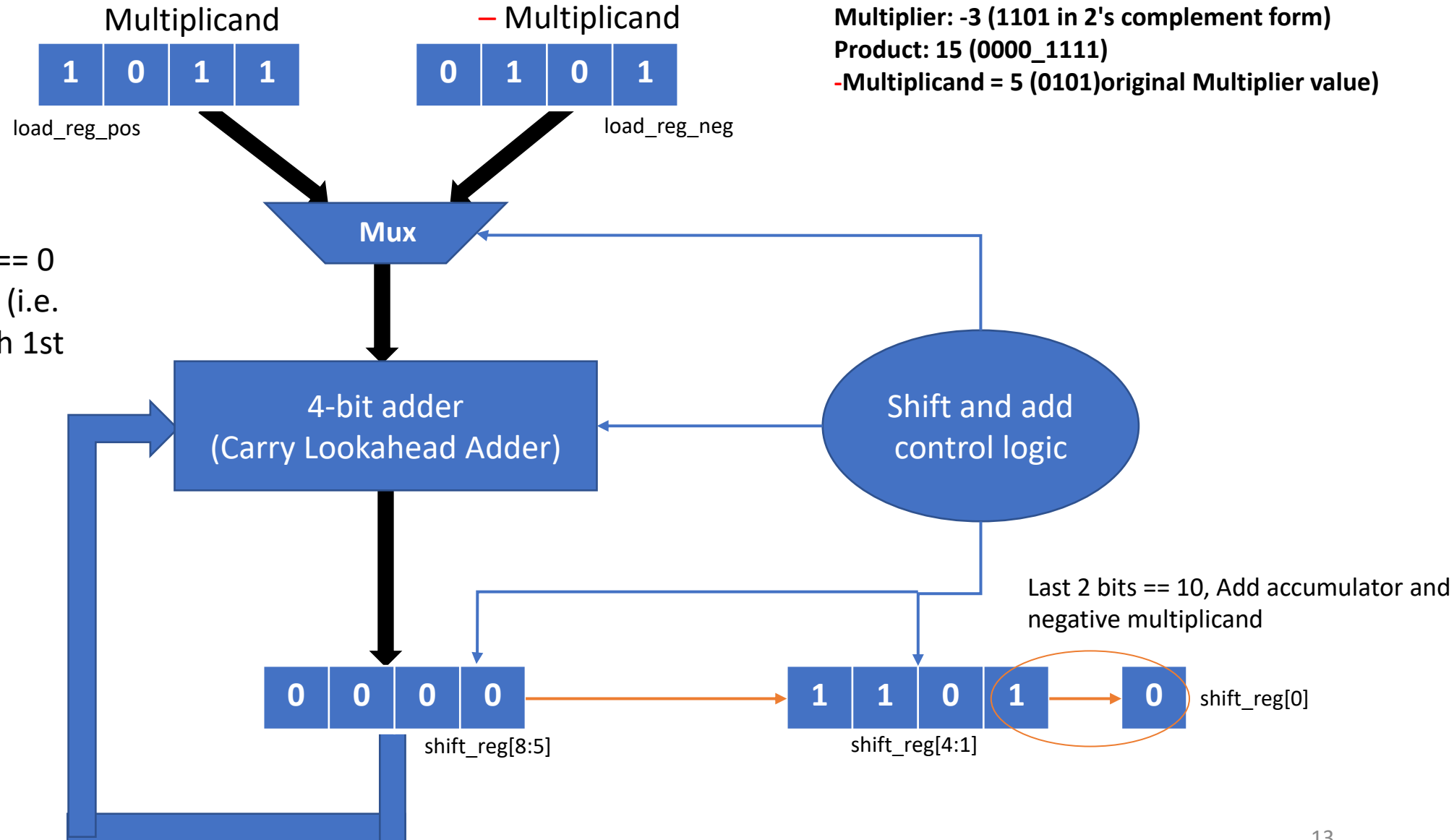


Booth Multiplier Algorithm Example Steps

**Stage Count-1 : Test
last 2 bits of shift
register**

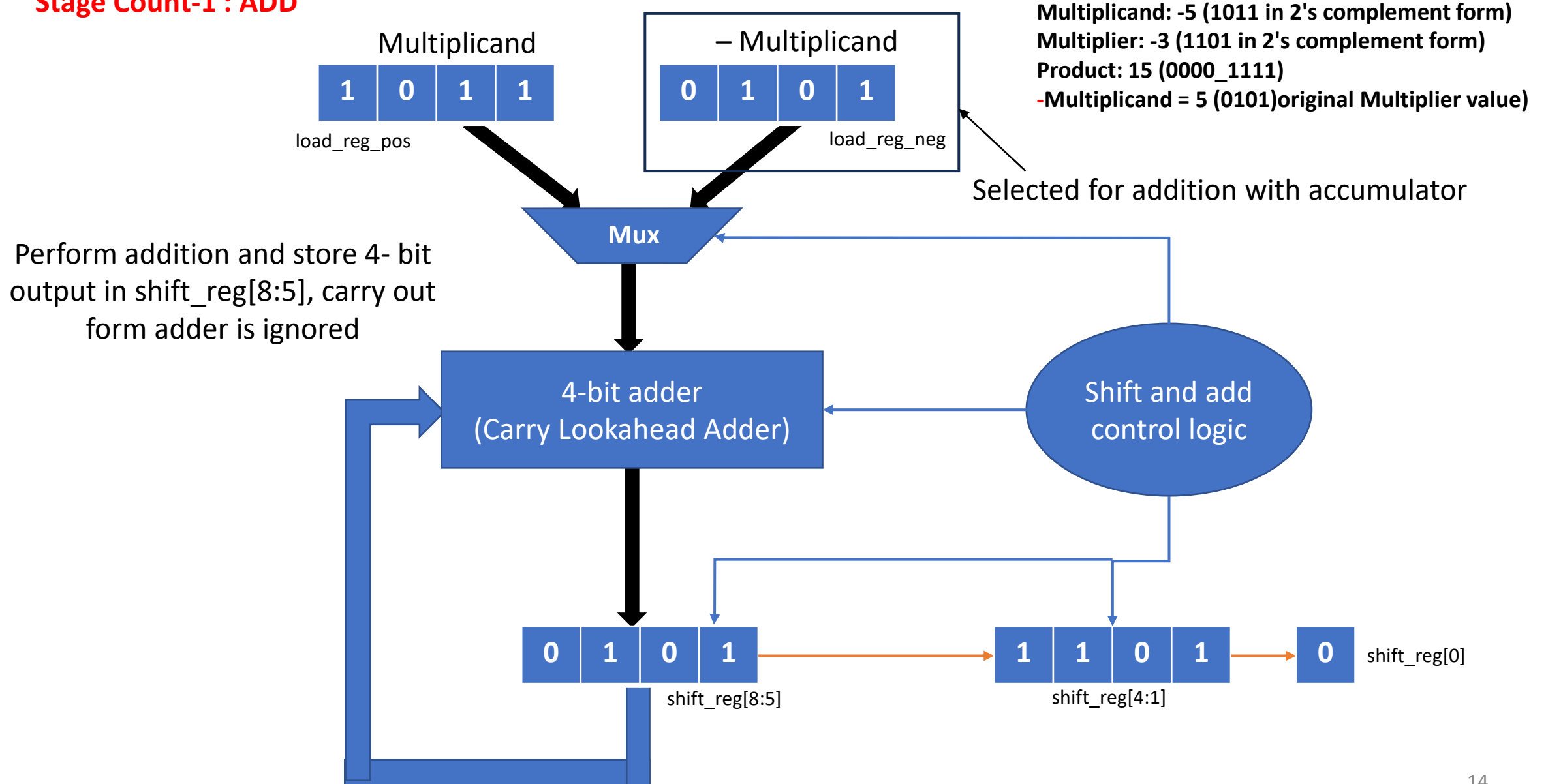
Multiplicand: -5 (1011 in 2's complement form)
Multiplier: -3 (1101 in 2's complement form)
Product: 15 (0000_1111)
-Multiplicand = 5 (0101)original Multiplier value)

Since stage count == 0
and not reached 3 (i.e.
N-1), continue with 1st
iteration



Booth Multiplier Algorithm Example Steps

Stage Count-1 : ADD



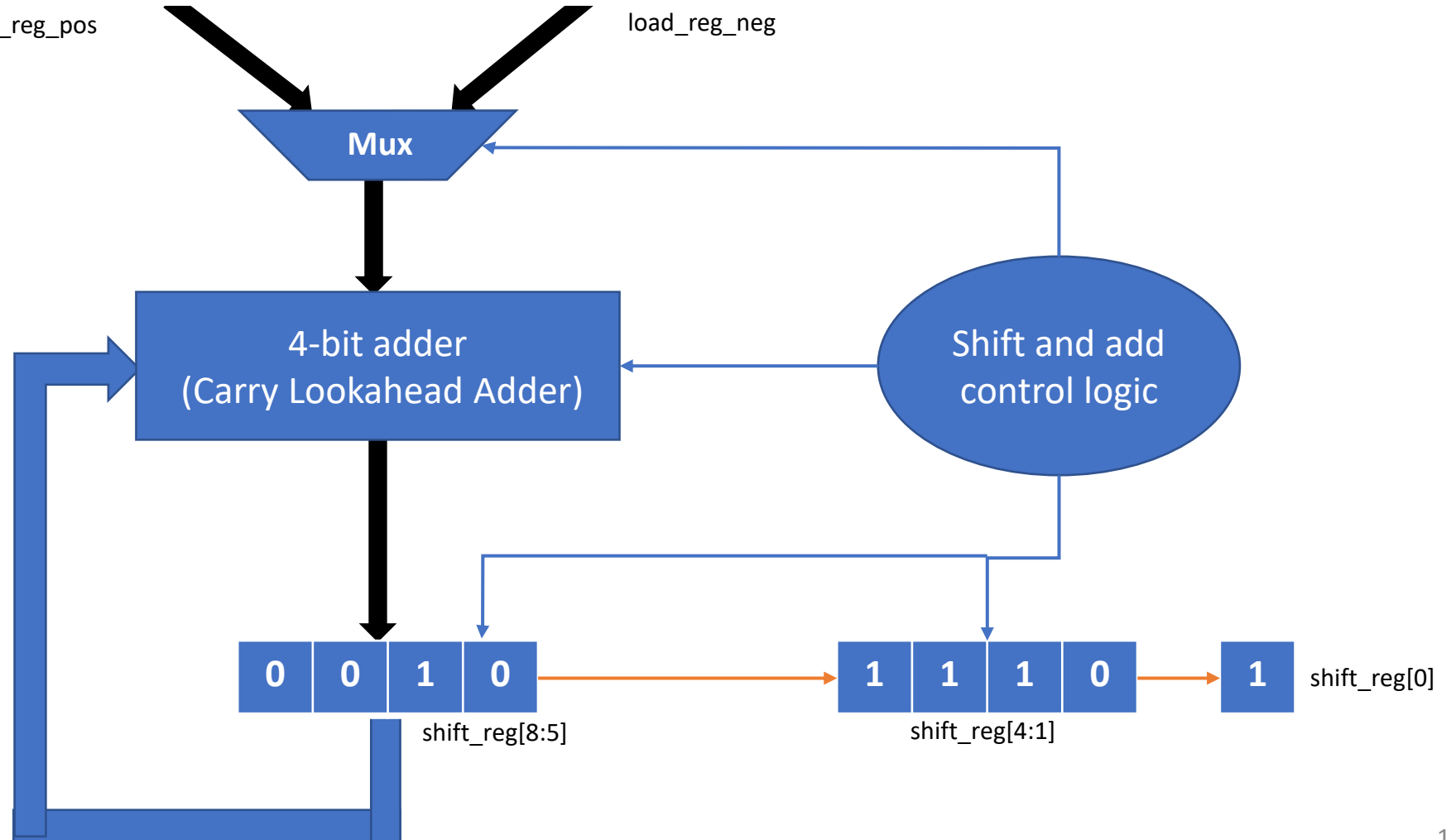
Booth Multiplier Algorithm Example Steps

Stage Count-1 : Shift >>>1



Multiplicand: -5 (1011 in 2's complement form)
Multiplier: -3 (1101 in 2's complement form)
Product: 15 (0000_1111)
-Multiplicand = 5 (0101)original Multiplier value)

Increment the value of
the Count register



*At the end of stage
count == 0, value of
shift_reg = 0010_1110_1

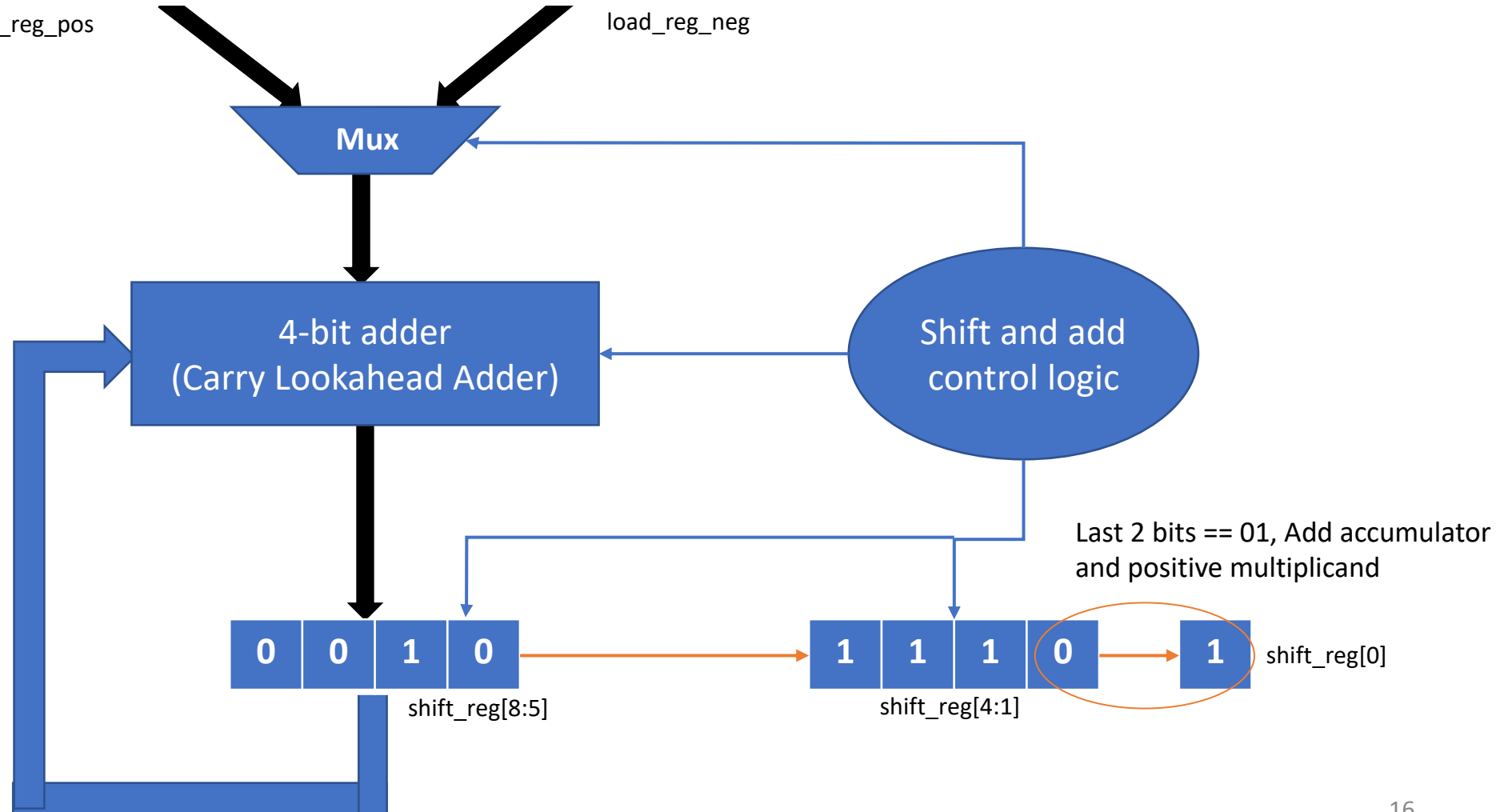
Booth Multiplier Algorithm Example Steps

Stage Count-2: Test
last 2 bits of shift_reg



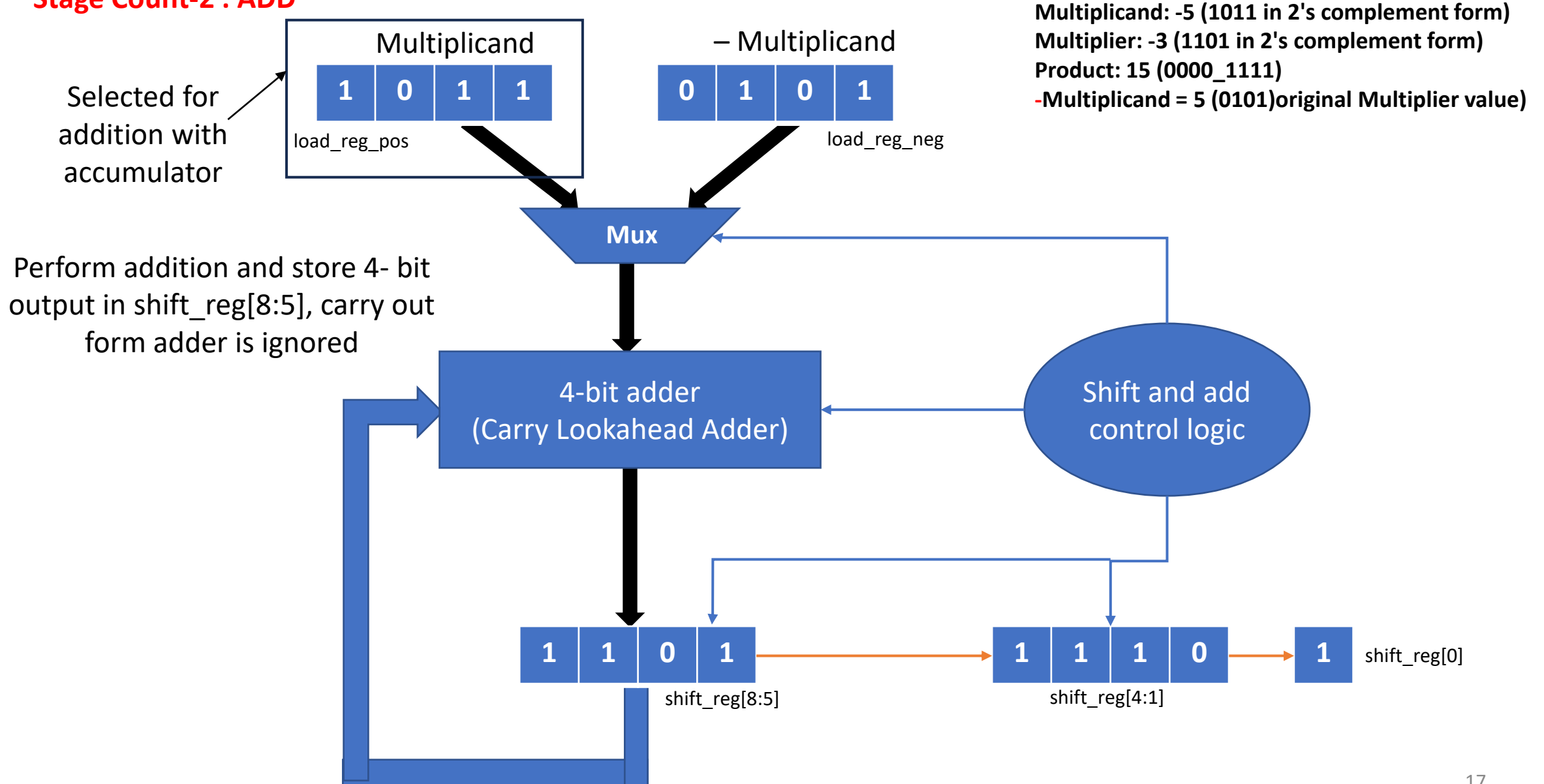
Multiplicand: -5 (1011 in 2's complement form)
Multiplier: -3 (1101 in 2's complement form)
Product: 15 (0000_1111)
-Multiplicand = 5 (0101) original Multiplier value)

Since stage count == 1
and not reached 3,
continue with 2nd
iteration



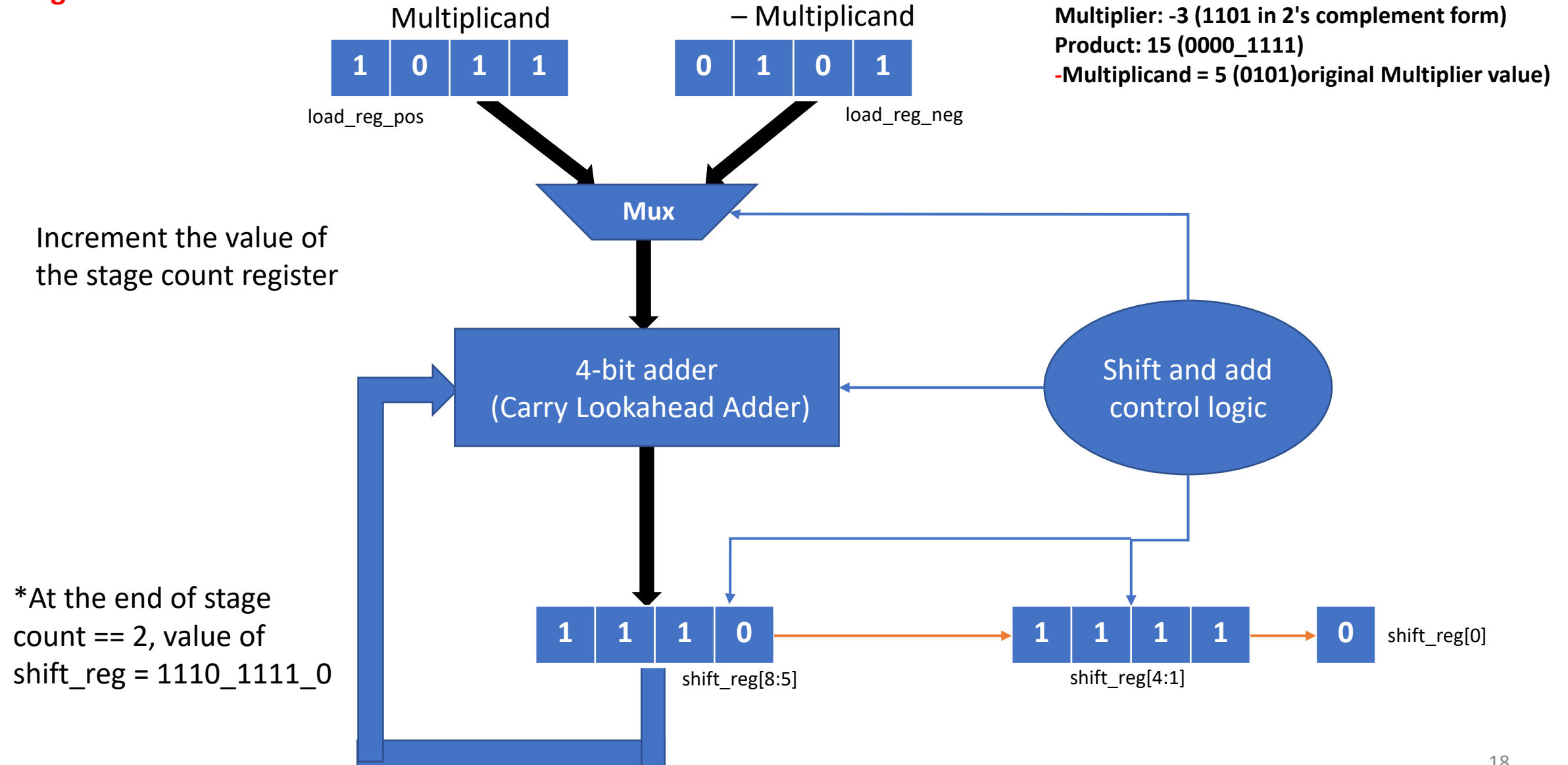
Booth Multiplier Algorithm Example Steps

Stage Count-2 : ADD



Booth Multiplier Algorithm Example Steps

Stage Count-2 : Shift >>>1



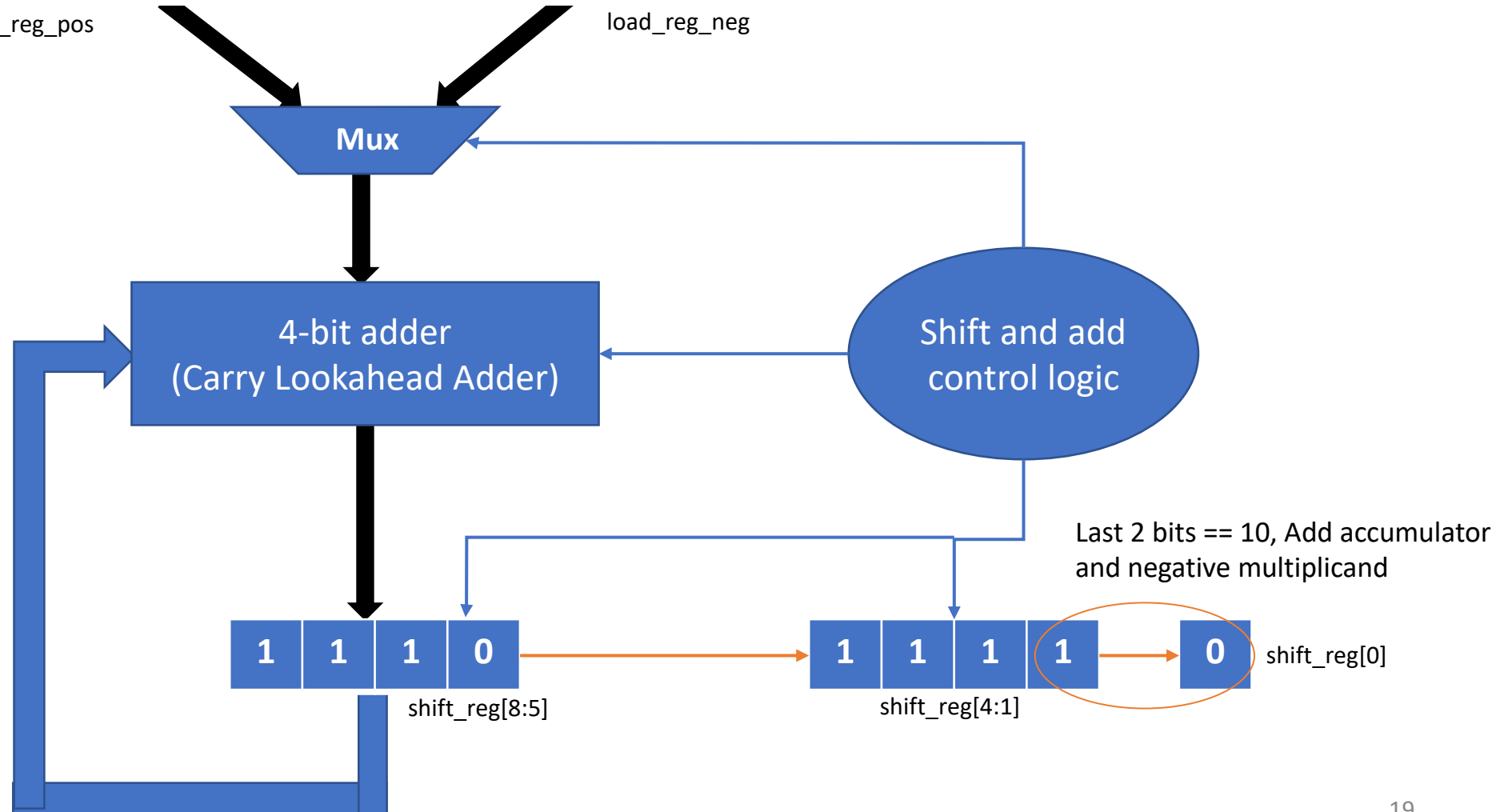
Booth Multiplier Algorithm Example Steps

Stage Count-3: Test
last 2 bits of shift_reg



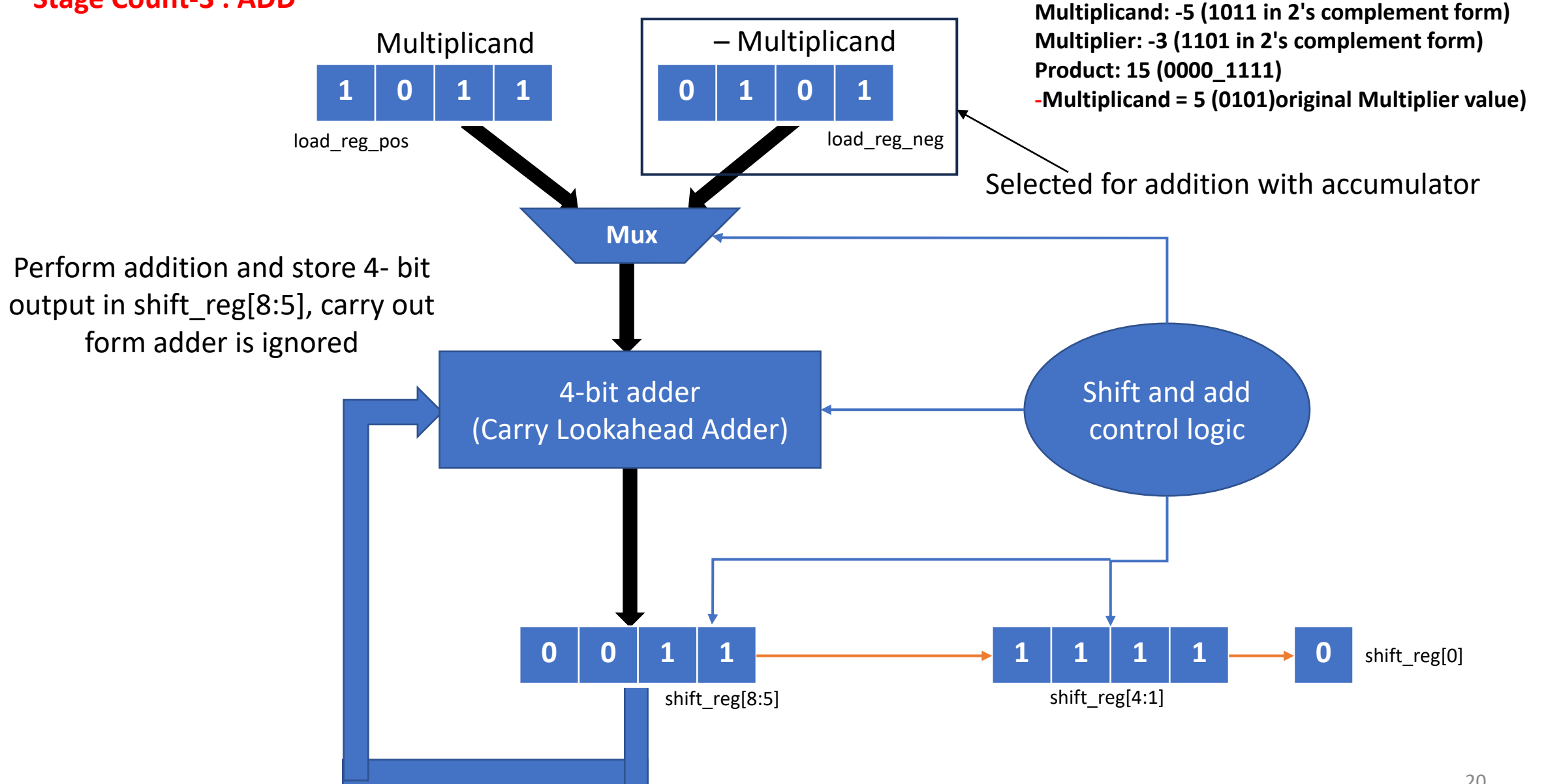
Multiplicand: -5 (1011 in 2's complement form)
Multiplier: -3 (1101 in 2's complement form)
Product: 15 (0000_1111)
-Multiplicand = 5 (0101)original Multiplier value)

Since stage count == 2
and not reached 3,
continue with 3rd
iteration



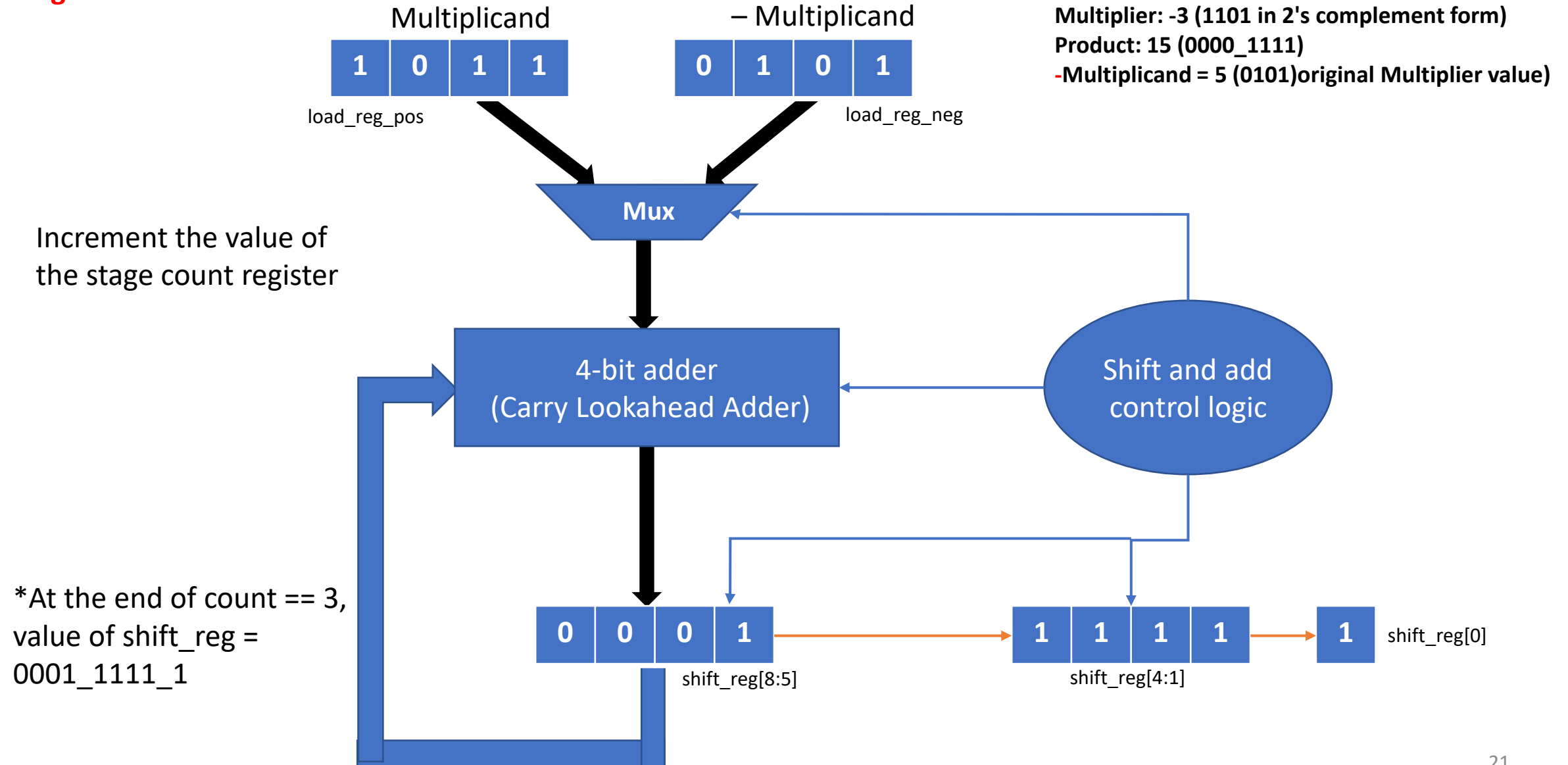
Booth Multiplier Algorithm Example Steps

Stage Count-3 : ADD



Booth Multiplier Algorithm Example Steps

Stage Count-3 : Shift >>>1

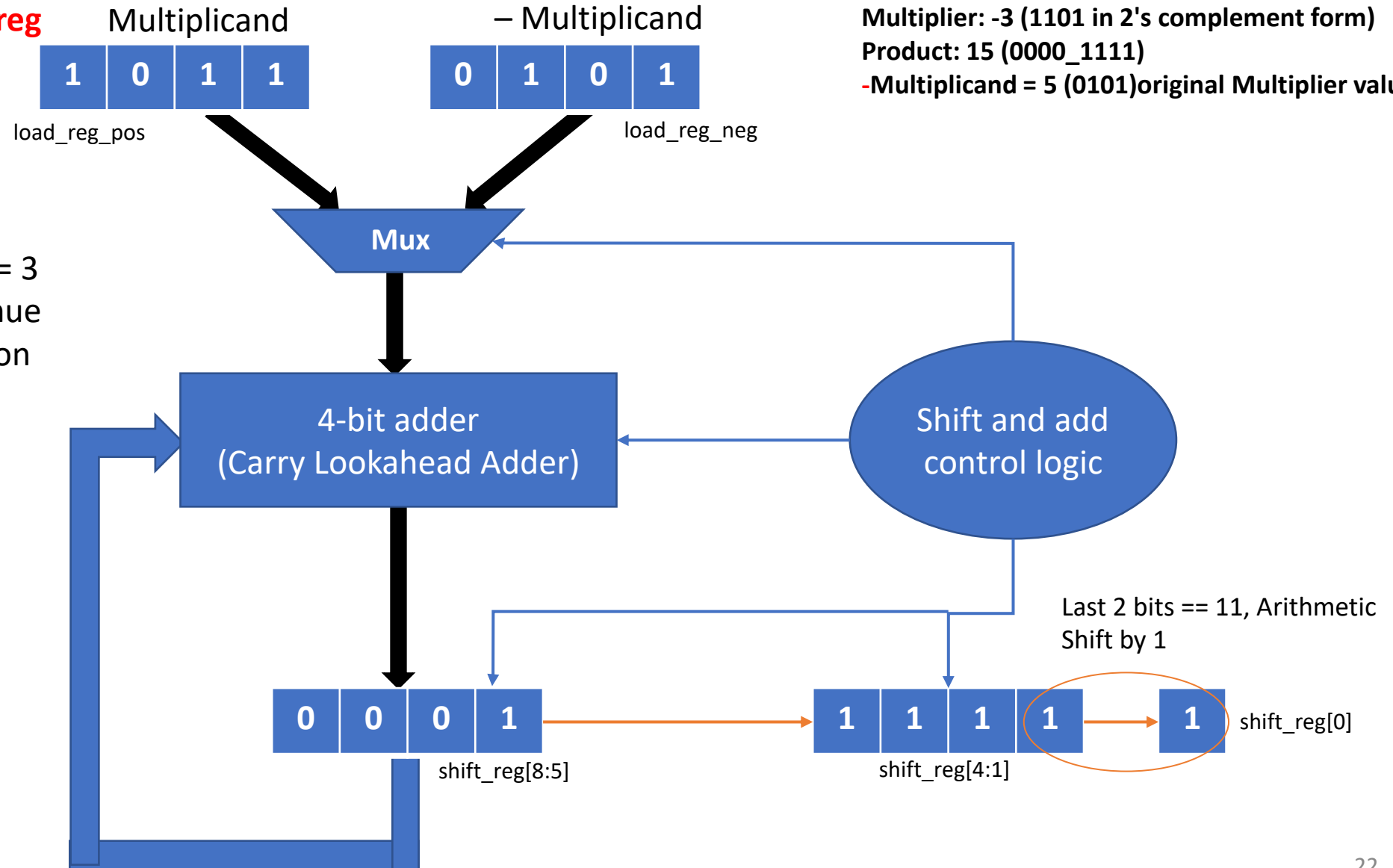


Booth Multiplier Algorithm Example Steps

**Stage Count-4: Test
last 2 bits of shift_reg**

Multiplicand: -5 (1011 in 2's complement form)
Multiplier: -3 (1101 in 2's complement form)
Product: 15 (0000_1111)
-Multiplicand = 5 (0101)original Multiplier value)

Since stage count == 3
and reached, continue
with final 4th iteration
and then stop



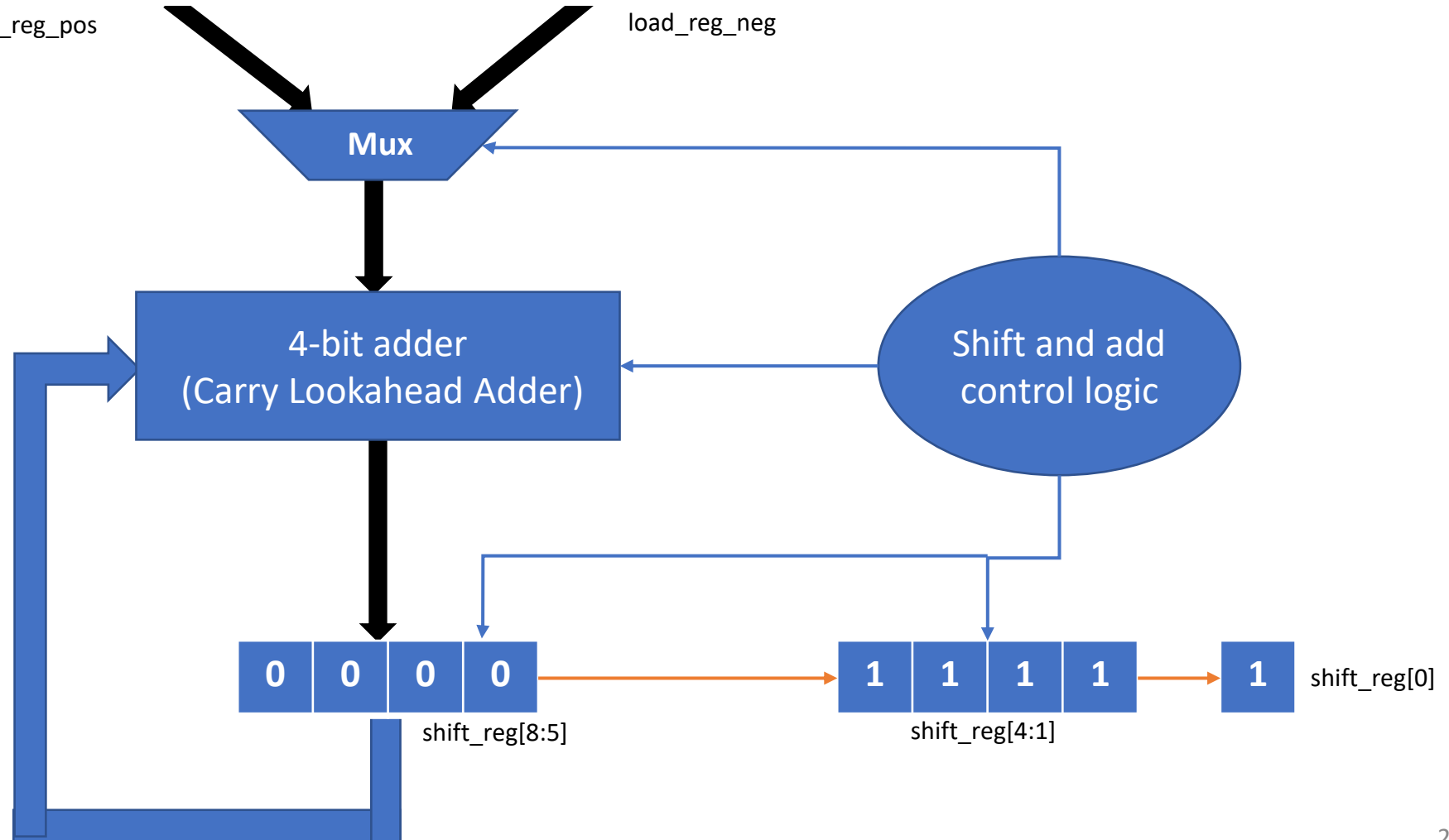
Booth Multiplier Algorithm Example Steps

Stage Count-4 : Shift >>>1



Multiplicand: -5 (1011 in 2's complement form)
Multiplier: -3 (1101 in 2's complement form)
Product: 15 (0000_1111)
-Multiplicand = 5 (0101) original Multiplier value)

Increment the value of the Count register



*At the end of stage count == 3, value of shift_reg = 0000_1111_1
Since stage count == 3 which is equal to 'N-1' value, final product value is generated using shift_reg[8:1] i.e.
8'b0000_1111 = 15

Integer Multiplier Code Development Hint

❑ Define 6 FSM States

```
// next_state encoding and next_state variable
enum logic[2:0]{
    IDLE          = 3'b000,
    INITIALIZE    = 3'b001,
    TEST          = 3'b010,
    ADD           = 3'b011,
    SHIFT_AND_COUNT = 3'b100,
    DONE          = 3'b101
} next_state;
```

❑ For N-bit Integer Multiplier, ensure size of shift register is = $N + N + 1$ bits

- N bits to store adder output without carry out in shift register (**this is also known as accumulator**)
- N bits to store multiplier value
- 1 extra bit required for Booth Algorithm
- shift register format = {N-bit Adder output, N-bit Multiplier, extra bit}

```
19 // Register to store Adder sum and multiplier
20 logic signed [(2*N):0] shift_reg;
```

❑ Use Carry Look Ahead Adder and Full Adder Module implementation from previous homework assignment

```
41 // Instantiate (N)-bit carry lookahead adder
42 carry_lookahead_adder #(N(N)) adder_inst(
43     // Student to add code
44     // Student to add code
45     // Student to add code
46 );
```

Integer Multiplier Code Development Hint

- ❑ Develop FSM code using `always_ff` block approach with non-blocking assignment statements

```
51 always_ff@(posedge clock, posedge reset) begin
52   if(reset) begin
53     count <= 0;
54     next_state <= IDLE;
55     load_reg_pos <= 0;
56     load_reg_neg <= 0;
57     shift_reg <= 0;
58   end
59   else begin
60     case(next_state)
61       // Wait for start signal
62       IDLE: begin
63         // Student to add code
64       end
65
66       // Load Multiplicand and Multiplier in a load register and a shift register
67       INITIALIZE: begin
68         // load multiplicand to load_reg_pos
69         // Student to Add Code Here
70
71         // load multiplicand_neg[N:0] to load_reg_neg
72         // Student to Add Code Here
73
74         shift_reg <= {{N{1'b0}}, multiplier, 1'b0};
75         next_state <= TEST;
76         count <= 0;
77       end
78     endcase
79   end
80 end
```

Booth Multiplier Code Development Hint

FSM code framework.... Continued

```
83 // Check shift register LSB and based on that perform ADD/Shift operation
84 // if last 2 LSB='01' then perform ADD with positive multiplicand followed by Airthmetic Right Shift by 1
85 // if last 2 LSB='10' then perform ADD with negative multiplicand followed by Airthmetic Right Shift by 1
86 // if last 2 LSB='00' then perform Right Shift by 1
87 // if last 2 LSB='11' then perform Right Shift by 1
88 TEST: begin
89     if(shift_reg[1:0] == 2'b01) begin
90         // Pass positive Multiplicand to carry lookadahead adder input
91         // Pass previous adder output value after shift to add with Multiplicand
92         // move to add state
93
94         // Student to add code here
95
96     end
97     else if(shift_reg[1:0] == 2'b10) begin
98         // Pass negative Multiplicand to carry lookadahead adder input
99         // Pass previous adder output value after shift to add with Multiplicand
100        // move to add state
101
102        // Student to Add code here
103
104    end
105    else begin
106
107        // assign add_operand1 to 0, Since no add operation to be perform pass 0 to carry lookadder input
108        // Pass previous adder output value after shift to add with Multiplicand
109        // move to shift and increment count state
110
111        // Student to Add code
112
113    end
114 end
115
116 ADD: begin
117     shift_reg <= {sum, shift_reg[N:0]}; // Load shift register : Output sum from Adder which includes carry and retain previous lower bit of shift register
118
119     // Move to shift and increment count state
120     // Studen to add code here
121
122 end
123
```

Booth Multiplier Code Development Hint

❑ FSM code framework.... Continued

```
25 SHIFT_AND_COUNT: begin
26     shift_reg <= (shift_reg >>> 1); // Right Arithmetic shift entire shift register by 1 position
27
28     // Increment count
29     // Student to Add code here
30
31
32     if(count == N-1) begin // If 'N' times SHIFT operation performed then move to Done state else go back to Test state
33         // Student to Add code here
34
35     end
36     else begin
37         // Student to Add code here
38
39     end
40
41 end
42 end
43
44 DONE: begin
45     next_state <= IDLE; // Wait for right shift value to be available. This is the final product value.
46 end
47 endcase
48 end
49 end
```

❑ Using assign statement generate output 'done' value when FSM transitions to DONE state

```
151 // Generate done=1 when FSM reaches DONE state
152 assign done = (next_state == DONE) ? 1 : 0;
```

Booth Multiplier Code Development Hint

- ❑ Using `always_comb` block generate "product" output value when FSM transitions to DONE state
 - In cases when multiplicand and multiplier both having their MSB bit's as 1, the final product value after all steps of booth multiplier algorithm are completed, and if the MSB bit of shift register is '1' which indicates negative result then 2's complement should be performed before generating final product value. Not doing so, will result in final product value generated with incorrect sign.
 - **Example :** In 4-bit binary system, $-8 \times -7 = 8'b1100_1000$ which is -56. After performing 2's complement it will result in $8'b0011_1000$ which is 56. And positive 56 is the correct product value for -8×-7

```
134 // Generate Product in DONE state by loading shift_reg value to it
135 always_comb begin
136     if(next_state == DONE) begin
137         if((shift_reg[(2*N)] == 1) && (multiplier[N-1] == 1) && (multiplicand[N-1] == 1)) begin
138             product = -shift_reg[(2*N):1];
139         end
140         else if((shift_reg[(2*N)] == 0) && (multiplier[N-1] == 0) && (multiplicand[N-1] == 1)) begin
141             product = -shift_reg[(2*N):1];
142         end
143         else begin
144             product = shift_reg[(2*N):1];
145         end
146     end
147 else begin
148     product = 0;
149 end
150 end
```


Booth Multiplier Code Development Hint

- ❑ Using `always_comb` block generate "product" output value when FSM transitions to DONE state
 - In cases when multiplicand has boundary condition negative value such as -8 in 4-bit binary system and multiplier has positive value, final product value will be generated with incorrect sign. To address this issue, 2's complement should be performed before generating final product value.
 - **Example** : $-8 \times 6 = 8'b0011_0000$ which is +48. After performing 2's complement it will result in $8'b1101_0000$ which is -48. And negative 48 is the correct product value for -8×6 .

```
134 // Generate Product in DONE state by loading shift_reg value to it
135 always_comb begin
136   if(next_state == DONE) begin
137     if((shift_reg[(2*N)] == 1) && (multiplier[N-1] == 1) && (multiplicand[N-1] == 1)) begin
138       product = -shift_reg[(2*N):1];
139     end
140     else if((shift_reg[(2*N)] == 0) && (multiplier[N-1] == 0) && (multiplicand[N-1] == 1)) begin
141       product = -shift_reg[(2*N):1];
142     end
143     else begin
144       product = shift_reg[(2*N):1];
145     end
146   end
147   else begin
148     product = 0;
149   end
150 end
```

References

❑ Booth Multiplier Calculator :

- <https://www.grahn.us/projects/booths-algorithm/>

❑ Booth Multiplier Algorithm :

- <https://www.geeksforgeeks.org/computer-organization-booths-algorithm/>